

PRAKTIKUM INFORMATIK 2

M. Farmbauer, M. Niemetz

Vererbung bei Klassen

Graphisches Schalterelement

Mai 2026, Version: 3.7

Vorbereitung

- ☐ ☕☕ **Syntax in UML-Klassendiagrammen:** Rekapitulieren Sie die Syntax der UML-Klassendiagramme, insbesondere die Bedeutung der unterschiedlichen Beziehungen („Pfeilarten“), die in diesen Diagrammen darstellbar sind. 3
- ☐ ☕ **Referenzen als Übergabeparameter / `const` für Parameter und Methoden:** Falls Ihnen die Vor- und Nachteile der Verwendung von Referenzen als Übergabeparameter sowie die Bedeutung der `const`-Eigenschaft für Methoden nicht (mehr) geläufig sind, lesen Sie nach! 4
- ☐ ☕ **Setzen der Füllfarbe bei WinAdapt.:** Schlagen Sie in der WinAdapt-Anleitung nach, wie Sie gefüllte Rechtecke zeichnen können und wie Sie die Füllfarbe von Rechtecken beeinflussen können. 5
- ☐ ☕ **Abgeleitete Klassen:** Lesen Sie ggf. nach, was abgeleitete Klassen sind, wozu sie dienen und wie Sie diese Ableitung in C++ spezifizieren können 6
- ☐ ☕ **Verwendung von Basisinitialisierern in der Initialisiererliste der abgeleiteten Klasse:** Lesen Sie nach, was Basisinitialisierer sind und wie sie in der Initialisiererliste des Konstruktors einer abgeleiteten Klasse zu verwenden sind. . 6
- ☐ ☕ **Namespace-Operator für globale Symbole verwenden:** Verwendung des Namespace-Operators zum Zugriff auf globale Symbole, die im lokalen Kontext verdeckt sind. 6
- ☐ ☕☕ **Virtuelle Funktionen:** Finden Sie heraus, was „virtuelle Funktionen“ sind (die Bezeichnung ist absolut irreführend!) und wodurch sie sich von nicht-virtuellen unterscheiden. . . 13

1 Einleitung

Für die Erstellung graphischer Benutzeroberflächen werden meist Bibliotheken mit vorgefertigten Kontrollelementen verwendet. Leider sind dabei typischerweise nicht alle benötigten Kontrollelemente verfügbar und es ist notwendig, für die eigene Anwendung spezielle Kontrollelemente zu erstellen.

In dieser Aufgabe werden exemplarisch zwei Kontrollelemente entwickelt, die dann in der nächsten Aufgabe verwendet werden.

Wahrscheinlich ist Ihnen bereits bei der vorangehenden Praktikumsaufgabe Wärmespeicher aufgefallen, dass die dort eingesetzten Klassen gewisse Gemeinsamkeiten aufweisen, was zu gewissen Redundanzen (kopierter Programmcode) in ihren Implementierungen führt.

Ein hierarchisches Softwaredesign ermöglicht es in solchen Situationen, die Gemeinsamkeiten in eine Basisklasse zu verschieben, wodurch die Redundanz beseitigt und damit der Codier- und Pflegeaufwand reduziert wird. Gleichzeitig sinkt das Risiko für Softwarefehler, da langweilige Routineaufgaben wie die Sicherstellung gleicher Implementierungen in verschiedenen Programmteilen vermieden werden.

In Abbildung 1 sehen Sie die Struktur des Projekts dieser Aufgabe dargestellt, die intensiv die Möglichkeiten der Vererbung nutzt und die Zusammenarbeit unterschiedlichster Klassen zur Erfüllung der Anforderungen verwendet.

Recherche ☕☕

Syntax in UML-Klassendiagrammen

Hinweis: Sie gewinnen inzwischen an Selbständigkeit. Daher enthalten die Anleitungen zunehmend weniger detaillierte Anweisungen sondern eher Zielvorgaben. Dennoch lohnt es sich, eine Teilaufgabe jeweils komplett zu lesen, um von eventuellen Hinweisen und Warnungen profitieren zu können.

Wichtig: Die Angabe enthält in den meisten Fällen keine Informationen zur korrekten Verwendung von **const**. Die müssen sich also bei der Implementierung selbst um die Herstellung der „const correctness“ kümmern.

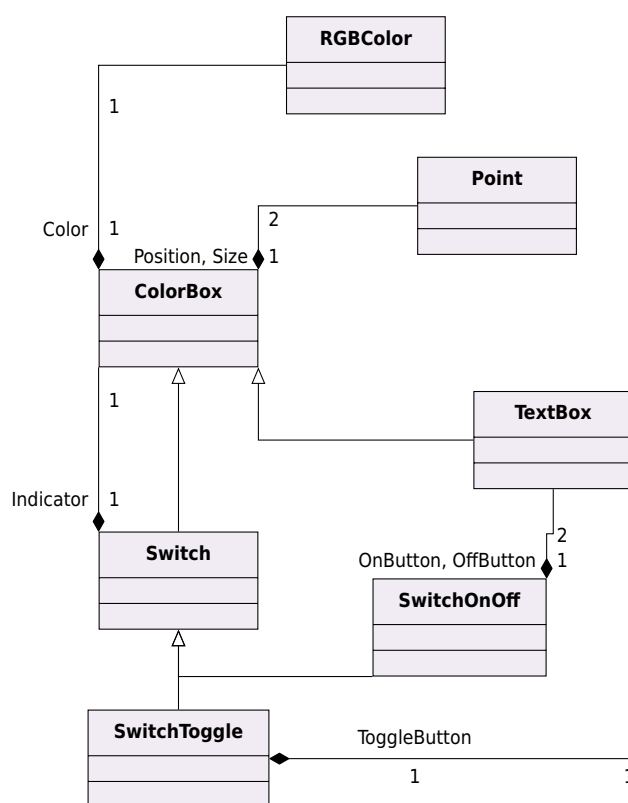


Abbildung 1: Darstellung der Architektur des Projekts bestehend aus den Klassen **Point**, **RGBColor**, **ColorBox**, **TextBox**, **Switch**, **SwitchOnOff** und **SwitchToggle** als UML Klassendiagramm.

2 Vorbereitung



Aktualisieren Sie ihren Fork des PIN2-Projekts, indem Sie `git pull upstream master` aufrufen.

Überprüfen Sie anschließend, ob sich das Verzeichnis für diese Aufgabe nun in Ihrem Repository befindet.

Wichtig: Verwenden Sie **immer konstante Referenzen** als Übergabeparameter, um unnötiges Kopieren und gleichzeitig unbeabsichtigte Veränderungen der Parameter zu vermeiden und maximale Kompatibilität beim Aufruf zu erreichen.

Verwenden Sie auch `const` Methoden, **wo immer dies sinnvoll ist**. Sie ersparen sich damit zeitintensive Nacharbeiten.



Aktualisieren Sie ihren Fork des PIN2-Projekts.

Recherche ☕

Referenzen als Übergabeparameter / `const` für Parameter und Methoden

2.1 Klasse Point

Point
+ X: double + Y: double
+ Point(X: double, Y: double) + operator+ (rhs: Point&): Point + operator- (rhs: Point&): Point + operator- (): Point + operator* (rhs: double): Point + operator/ (rhs: double): Point + operator% (rhs: double): Point + operator+= (rhs: Point&): Point& + operator-= (rhs: Point&): Point& + operator*= (rhs: double): Point& + operator/= (rhs: double): Point& + operator<(rhs: Point&): bool + operator<=(rhs: Point&): bool + operator>(rhs: Point&): bool + operator>=(rhs: Point&): bool + operator!=(rhs: Point&): bool + operator==(rhs: Point&): bool + operator<(rhs: double): bool + operator<=(rhs: double): bool + operator>(rhs: double): bool + operator>=(rhs: double): bool + abs(): double

Für die grafische Darstellung werden immer wieder Koordinaten von Bildschirmpunkten benötigt.

Hierfür enthält der Programmrahmen bereits eine Implementierung der Ihnen aus den vergangenen Aufgaben wohlbekannten Klasse `Point`. Sie sollten möglichst alle Berechnungen über Objekte dieser Klasse durchführen und nur dann komponentenweise Operationen verwenden, wenn dies (z.B. aufgrund der Schnittstelle zu WinAdapt) unvermeidbar ist.

Machen Sie sich daher nochmals mit den zur Verfügung stehenden Operatoren und Funktionen sowie deren Einsatzmöglichkeiten für Berechnungen mit Bildschirmkoordinaten vertraut.



Machen Sie sich mit den Einsatzmöglichkeiten der Klasse `Point` vertraut.

2.2 Klasse RGBColor

RGBColor
+ R: int + G: int + B: int
+ RGBColor(R: int, G: int, B: int) + operator ==(p: RGBColor&): bool

Für die Handhabung von Farben in Ihren graphischen Darstellungen benötigen Sie eine Klasse zur Arbeit mit Farben. Eine sehr einfache solche Klasse liegt dem Programmrahmen in Gestalt der Klasse `RGBColor` bei.



Analysieren Sie die Klasse `RGBColor`

Analysieren Sie diese Klasse `RGBColor` zur Speicherung eines Farbwertes im RGB-Farbraum.

3 Implementierung der Klassen

Gemäß der Übersicht in Abbildung 1 sind einige (aber dafür nicht sehr umfangreiche) Klassen zu implementieren.

3.1 Klasse ColorBox

ColorBox
<pre># Position : Point # Size : Point # Color : RGBColor</pre>
<pre>+ ColorBox(Position: Point&, Size: Point&, Color: RGBColor&) + contains(f: Point&): bool + getPosition(): Point + getSize(): Point + getColor(): RGBColor + setPosition(p: Point&): void + setSize(s: Point&): void + setColor(c: RGBColor&): void + show(): void</pre>

Erstellen Sie die Klasse `ColorBox` in einer neuen .h-Datei `ColorBox.h` und implementieren Sie die Funktionen der Klasse entsprechend der Angaben im nebenstehenden UML-Klassendiagramm in einer entsprechenden .cpp-Datei `ColorBox.cpp`.

Die Datenelemente der Klasse `ColorBox` sollen alle **protected** sein, wie im Klassendiagramm am Zeichen **#** zu erkennen ist. Damit trotzdem von

außen geordnet auf die Elemente zugegriffen werden kann, werden für den Lese- und Schreibzugriff jeweils entsprechende Zugriffsfunktionen bereitgestellt. Diese Funktionen sollen zunächst nur die Datenelemente auf die Werte der übergebenen Parameter setzen.

Darüber hinaus erhält die Klasse einen Konstruktor, der es gestattet, die Position (linke obere Ecke der Box), die Größe und die Füllfarbe bereits bei der Konstruktion festzulegen.

Wichtig: Verwenden Sie im Konstruktor unbedingt die Initialisierersliste um die Konstruktion der Datenelemente zu steuern, sonst wird Ihr Code vom Compiler nicht akzeptiert!

Eine `show`-Methode und die Methode `contains`, welche jeweils die selben Aufgaben haben, wie aus den letzten Aufgaben um die Klassen des Wärmespeichers bekannt, komplettieren die Funktion der Klasse.

Die Methode `show` soll dabei ein farbig gefülltes Rechteck entsprechend der Farb- und Größen- und Positionsdaten des Objekts zeichnen.

Die Funktion `contains` soll im Rückgabewert angeben, ob die als Parameter übergebenen Koordinaten auf dem Bildschirm innerhalb der `ColorBox` liegen.

Testen Sie nun Ihre neue Implementierung der Klasse `ColorBox` mit dem `TEST_LEVEL TEST_COLORBOX`.



Erzeugen Sie nach dem erfolgreichen Test eine neue Revision und laden Sie das Ergebnis auf den Kursserver hoch.



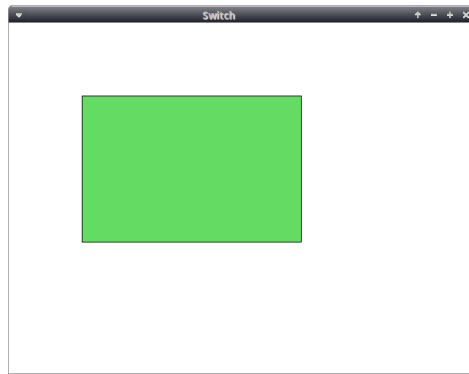
Erstellen Sie die Klasse `ColorBox`.

Recherche ☕

Setzen der Füllfarbe bei WinAdapt.



Testen Sie die Klasse `ColorBox`. Verwenden Sie dazu die Einstellung `TEST_LEVEL TEST_COLORBOX`.

Abbildung 2: Bildschirmausgabe im Modus `TEST_COLORBOX`.

3.2 Klasse TextBox

TextBox
- Text : string
+ TextBox(Position: Point&, Size: Point&, Text: string&) + getText(): string + setText(Text: string&): void + show(): void

Erstellen Sie nun die neue Klasse `TextBox` mit dem im nebenstehenden UML-Diagramm angegebenen Datenelement sowie den dort gelisteten Funktionen. Legen Sie die hierfür nötigen Dateien `TextBox.h` und `TextBox.cpp` an. Beachten Sie, dass `TextBox` eine Ableitung von der Klasse `ColorBox` sein soll und diese

um die Möglichkeit erweitert, eine Beschriftung anzuzeigen.

Die aus den vorangehenden Aufgaben bekannte `string`-Klasse wird wieder zur komfortablen Verwaltung der Zeichenkette `Text` verwendet.

Hinweis: Im Konstruktor müssen die von der Basisklasse geerbten Datenelemente ebenfalls initialisiert werden. Verwenden Sie einen Basisinitialisierer um die Konstruktorimplementierung der Basisklasse hierfür zu nutzen!

Diese „Initialisierung der Basisklasse“ funktioniert dabei analog zur Initialisierung der eigenen Datenelemente. Wählen Sie für die Initialisierung der Basisklasse eine geeignete Farbe zur Übergabe an den Konstruktor aus.

Die `show`-Methode der Klasse `TextBox` zeichnet ein farbiges Rechteck mit Beschriftung. Dafür wird zur Darstellung des farbigen Rechtecks die geerbte `show`-Methode der Basisklasse aufgerufen. Danach wird der Text „über“ das farbiges Rechteck gedruckt.

Wichtig: Der Name der globalen `WinAdapt`-Funktion zur Ausgabe des Texts lautet genauso, wie das Datenelement, welches den Text speichert. Dadurch wird der Name der globalen Funktion `Text` innerhalb der Methoden der Klasse *verdeckt* und ist nicht direkt zugreifbar. Verwenden Sie den bekannten Operator `::` um auf die globale Funktion `Text` zuzugreifen. Der Name des globalen Namensraums, in dem die Funktion `Text` von `WinAdapt` lebt, ist leer.



Erstellen Sie die Klasse `TextBox` als Spezialisierung der Klasse `ColorBox`.

Recherche ☕

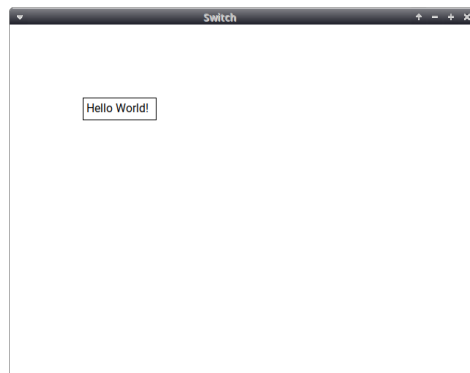
Abgeleitete Klassen

Recherche ☕

Verwendung von Basisinitialisierern in der Initialisiererliste der abgeleiteten Klasse

Recherche ☕

Namespace-Operator für globale Symbole verwenden

Abbildung 3: Bildschirmausgabe im Modus `TEST_TEXTBOX`.

Testen Sie nun Ihre neue Implementierung der Klasse `TextBox` mit dem `TEST_LEVEL TEST_TEXTBOX`.



Erzeugen Sie nach dem erfolgreichen Test eine neue Revision und laden Sie das Ergebnis auf den Kursserver hoch.



Testen Sie die Klasse `TextBox`. Verwenden Sie dazu die Einstellung `TEST_LEVEL TEST_TEXTBOX`

3.3 Klasse Switch

Switch
State : bool # Indicator: ColorBox*
+ Switch() + Switch(Source: Switch&) + Switch(Position: Point&) + ~Switch() + onMouse(Position: Point&): void + setPosition(Position: Point&): void + getState(): bool + setState(State: bool): void + operator= (rhs: Switch&): Switch& + show(): void

Erstellen Sie nun die Klasse `Switch` mit den im nebenstehenden UML-Diagramm angegebenen Attributen und Methoden.

Diese Klasse soll als universelle Basisklasse für unterschiedliche Schaltertypen dienen. Daher besitzt sie zwar einen Schaltzustand und einen Indikator, der den aktuellen Schaltzustand auf dem Bildschirm sichtbar macht, jedoch noch keine Be-

dienelemente, die es dem Nutzer des Programms gestatten würden, den Schaltzustand zu beeinflussen. Legen Sie für die Definition der Klasse und die Implementierung der Funktionen die Dateien `Switch.h` und `Switch.cpp` an.

Die Abbildung 4 zeigt die möglichen Zustände des Basischalterelements: Das Statusanzeigeelement zeigt je nach Schaltzustand die Farbe grün (Ein) oder rot (Aus).

Bei der Implementierung der Klasse `Switch` gilt es folgende Punkte zu beachten:

- Der Schalter soll eine Höhe von 36 Pixeln und eine Breite vom 102 Pixeln besitzen.
- Die Hintergrundfarbe des Schalters soll grau sein.



Erstellen Sie die Klasse `Switch`.

- Der Indikator ist ein per Pointer referenziertes `ColorBox`-Objekt und soll 30 mal 30 Pixel groß und drei Punkte vom oberen unteren und rechten Rand des Schalters entfernt sein. Die anfängliche Füllfarbe soll dem in `State` gespeicherten Zustand des Schaltelements entsprechend rot (`false`) oder grün (`true`) sein.
- Die Funktion `setPosition` soll nicht nur den Schalter sondern auch das Indikatorelement korrekt positionieren.
- Der Konstruktor muss alle Datenelemente initialisieren. Verwenden Sie ausschließlich die Initialisiererliste hierfür!
Hinweis: Bedenken Sie, dass Sie wieder den Basisinitialisierer in der Initialisiererliste benötigen.
- Der Destruktor muss das Indikator-Objekt als einziges Eigentum des Schalters löschen.
- Die `onMouse`-Methode hat noch keine Funktion und ist daher die leere Funktion.
- Die Funktion `setState` soll entsprechend des Werts des Parameters das Datenelement `State` sowie die Farbe des Indikators, nämlich rot (`false`) oder grün (`true`), setzen.
- Die `show`-Funktion zeichnet zunächst den Rahmen im Hintergrund mittels Aufruf der `show`-Methode der Basisklasse und ruft dann die `show`-Methode des von `Indicator` referenzierten Elements auf.
- Beim Copy-Konstruktor ist darauf zu achten, das neue Objekt als Kopie des Originals zu erzeugen. Dabei solle die Initialisiererliste, und darin insbesondere der Basisinitialisierer verwendet werden. Selbstverständlich ist hier eine tiefe Kopie des Indikator-Objekts notwendig.
- Beim Zuweisungsoperator ist analog vorzugehen, wie beim Copy-Konstruktor. Allerdings ist zusätzlich zu beachten, dass das `this`-Objekt bereits vorher ein gültiges Indikatorobjekt besessen hat.

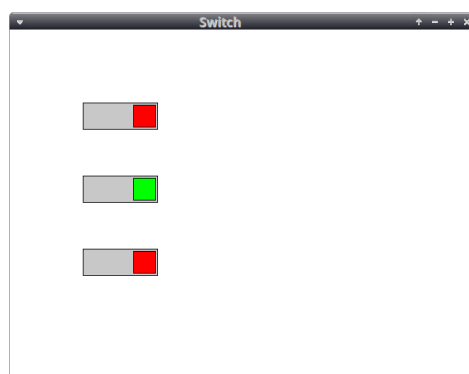


Abbildung 4: Bildschirmanzeige im Testlevel `TEST_SWITCH`: Dargestellt sind drei Objekte. Die Farbe des Indikators wird entsprechend dem in `State` gespeicherten Zustand des Schaltelements entsprechend umgeschaltet. Beachten Sie das als grauer Hintergrund sichtbare `ColorBox`-Objekt hinter dem Anzeigeelement und die verschiedenen Farben der Statusanzeige.

Testen Sie nun Ihre neue Implementierung der Klasse `Switch` mit dem `TEST_LEVEL TEST_SWITCH`.

Das Ergebnis sollte aussehen wie in Abbildung 4 dargestellt.



Erzeugen Sie nach dem erfolgreichen Test eine neue Revision und laden Sie das Ergebnis auf den Kursserver hoch.



Testen Sie die Klasse `Switch`. Verwenden Sie dazu die Einstellung `TEST_LEVEL TEST_SWITCH`

3.4 Klasse SwitchOnOff

SwitchOnOff
OnButton: TextBox* # OffButton: TextBox*
+ SwitchOnOff(s: SwitchOnOff&) + SwitchOnOff(pos: Point&, OnButtonLabel: string&, OffButtonLabel: string&) + ~SwitchOnOff() + onMouse(p: Point&): void + setPosition(p: Point&): void + operator= (s: SwitchOnOff&): SwitchOnOff& + show(): void

Nun fehlen noch Schalter mit den nötigen Bedienelementen. Erstellen Sie nun hierfür die Klasse `SwitchOnOff` mit den im nebenstehenden UML-Diagramm angegebenen Datenelementen und Funktionen. Legen Sie für die Definition der Klasse und die Implementierung der Funktionen die Dateien `SwitchOnOff.h` und `SwitchOnOff.cpp` an.



Erstellen Sie die Klasse `SwitchOnOff`.

Dieser Schalter soll nun das Umschalten des Zustands über zwei Bedienelemente erlauben, wobei eines für das Einschalten, das andere für das Ausschalten des Schaltelements zuständig ist. Die beiden Elemente werden über jeweils ein `TextBox`-Objekt realisiert, das über einen Pointer referenziert wird.

Dementsprechend sind folgende Details bei der Implementierung der Klasse zu beachten:

- Die beiden Parameter `OnButtonLabel` und `OffButtonLabel` des Konstruktors sollen die Defaultwerte 1 bzw. 0 erhalten.

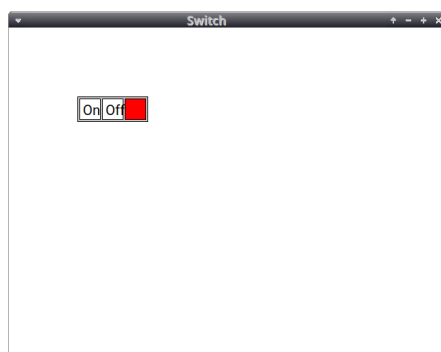


Abbildung 5: Aussehen des Schalters mit zwei Schaltelementen. Im Bild ist der Zustand im Modus `TEST_SWITCH_ONOFF` dargestellt.

- Die Positionen der beiden `TextBox`-Elemente `OnButton` und `OffButton` sowie das `ColorBox`-Element `Indicator` müssen im Konstruktor so festgelegt werden, dass mindestens 3 Pixel Abstand zueinander und zum Rand des Schalters vorhanden sind.
- Der Destruktor löscht die von `OnButton` und `OffButton` referenzierten Objekte.
- Der Copy-Konstruktor kopiert die von `OnButton` und `OffButton` referenzierten Objekte tief, wie auch der Zuweisungsoperator, der sich jedoch zusätzlich noch um die „alten“ Objekte kümmern muss, auf die die Pointer vorher gezeigt haben. Beide müssen sich dabei auch um die übrigen Datenobjekte kümmern, wobei der Zuweisungsoperator der Basisklasse hilfreich ist und auch genutzt werden soll.
- Die `onMouse`-Methode überprüft durch geeignete Aufrufe der Methoden dieser Objekte, ob die als Argument übergebene Position innerhalb des `OnButton`- oder des `OffButton`-Elements liegt und schaltet dann ggf. den `Status` durch Aufruf von `setState` entsprechend.
- Beim Versetzen des Schalters durch den Aufruf der `setPosition`-Methode müssen auch die im Schalter verwendeten Elemente korrekt verschoben werden.
- Die `show`-Methode zeichnet zunächst den Rahmen im Hintergrund mittels Aufruf der `show`-Methode der Basisklasse und ruft dann die `show`-Methoden der Elemente `OnButton` und `OffButton` auf.

Testen Sie nun Ihre neue Implementierung der Klasse `SwitchOnOff` mit dem `TEST_LEVEL TEST_SWITCH_ONOFF`.



Erzeugen Sie nach dem erfolgreichen Test eine neue Revision und laden Sie das Ergebnis auf den Kursserver hoch.



Testen Sie die Klasse `SwitchOnOff`. Verwenden Sie dazu die Einstellung `TEST_LEVEL TEST_SWITCH_ONOFF`.

3.5 Klasse SwitchToggle

SwitchToggle
ToggleButton: TextBox*
+ SwitchToggle(s: SwitchToggle&) + SwitchToggle(pos: Point&, ToggleButtonLabel: string&) + ~SwitchToggle() + onMouse(p: Point&): void + setPosition(p: Point&): void + operator= (s: SwitchToggle&): SwitchToggle& + show(): void

Nun fehlen noch Schalter mit den nötigen Bedienelementen. Erstellen Sie nun hierfür die Klasse `SwitchToggle` mit den im nebenstehenden UML-Diagramm angegebenen Datenelementen und Funktionen. Legen Sie für die Definition der Klasse und die Implementierung der Funktionen die Dateien `SwitchToggle.h` und `SwitchToggle.cpp` an.



Erstellen Sie die Klasse `SwitchToggle`.

Dieser Schalter soll das Umschalten des Zustands über ein Bedienelement erlauben, wobei dieses jeweils den Schaltzustand invertiert. Das Element wird wieder über ein `TextBox`-Objekt realisiert, das über einen Pointer referenziert wird.

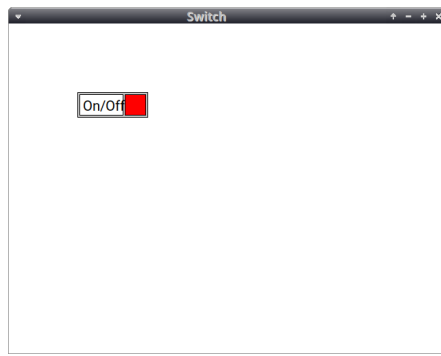


Abbildung 6: Aussehen des Schalters mit einem Ein-/Aus-Schaltelement. Im Bild ist der Zustand im Modus `TEST_SWITCH_TOGGLE` dargestellt.

Dementsprechend sind folgende Details bei der Implementierung der Klasse zu beachten:

- Der Parameter `ToggleButtonLabel` des Konstruktors soll den Defaultwert 1/0 erhalten.
- Die `onMouse`-Methode überprüft durch geeigneten Aufrufe der Methode des `ToggleButton`-Objekts, ob die als Argument übergebene Position innerhalb des `ToggleButton`-Objekts liegt und schaltet dann ggf. den `Status` durch Aufruf von `setState` und `getState` um.
- Alle übrigen Dinge sind analog zur Klasse `SwitchOnOff` umzusetzen.

Testen Sie nun Ihre neue Implementierung der Klasse `SwitchToggle` mit dem `TEST_LEVEL TEST_SWITCH_TOGGLE`.



Erzeugen Sie nach dem erfolgreichen Test eine neue Revision und laden Sie das Ergebnis auf den Kursserver hoch.



Testen Sie die Klasse `SwitchToggle`. Verwenden Sie dazu die Einstellung `TEST_LEVEL TEST_SWITCH_TOGGLE`.

4 Eine Taster-Klasse

Die beiden bisherigen Schaltertypen können die beiden statischen Zustände „Ein“ und „Aus“ darstellen. Für manche Anwendungen ist es aber vorteilhaft, einen Taster bzw. Momentkontakt zur Verfügung zu haben, der am Ende der Betätigung automatisch in den Ruhezustand zurückkehrt.

Sie sollen nun eine Klasse entwerfen und implementieren, welche einen solchen Taster bereitstellt. Damit Ihre Klasse kompatibel zum bereitgestellten Testprogramm ist, muss sie folgende Anforderungen erfüllen:

Req^I Die Klasse muss den Namen `SwitchPulse` tragen und in den Dateien `SwitchPulse.h` und `SwitchPulse.cpp` implementiert sein.

Req^{II} Die Klasse muss die Schnittstellen der Klasse `Switch` umsetzen.



Entwerfen und implementieren Sie eine Klasse für einen Taster.

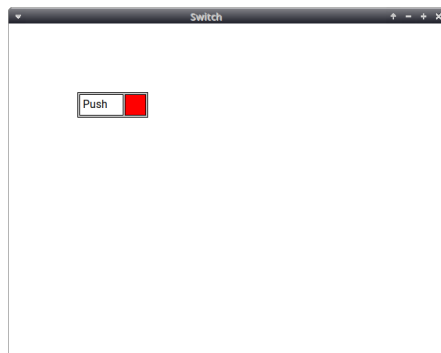


Abbildung 7: Mögliches Aussehen des Tasters mit Momentkontakt. Im Bild ist der Zustand im Modus `TEST_SWITCH_PULSE` dargestellt.

- Req^{III} Es muß ähnlich wie bei `SwitchToggle` ein Bedienfeld mit Beschriftung sowie einen farbigen Indikator geben, welcher den aktuellen Schaltzustand anzeigt. Weitere Vorgaben existieren nicht.
- Req^{IV} Ein Betätigen des Tasters muss den Zustand auf „Ein“ bringen und die interne „Uhr“ des Bausteins auf den Startzeitpunkt stellen. Ist der Taster bereits im Zustand „Ein“, so bleibt dies so, nur die Uhr wird zurückgestellt auf den Startzustand (also der Einschaltzustand wird gewissermaßen verlängert).
- Req^V Ist die Uhr abgelaufen, so wechselt der Taster wieder in den Zustand „Aus“.
- Req^{VI} Die Elementfunktion `onTimerTick`, die weder Rückgabewert noch Parameter besitzen soll, lässt den Timer um einen Schritt ablaufen und ändert bei Bedarf den Zustand des Tasters.
- Req^{VII} Es gibt einen Konstruktor, der neben den aus den vorangehenden Teilaufgaben schon bekannten Parametern `Position` und `Label` noch den zusätzlichen neuen Parameter `DecayTime` des Typs `int` besitzt. Sowohl `Label` wie auch `DecayTime` sollen optional sein. Der Default-Wert für `DecayTime` soll 10 betragen.

Hinweis: Es gibt mehrere Möglichkeiten, diese Teilaufgabe umzusetzen. Überlegen Sie, welche Optionen sich Ihnen bieten und wählen Sie dann das Design, welches Ihnen persönlich am vorteilhaftesten erscheint.

Frage: Wie sieht Ihre Klasse aus? Entwerfen Sie ein Klassendiagramm und zeichnen Sie ggf. die wichtigsten Verbindungen zu bereits bestehenden Klassen ein.

Implementieren Sie nun die Klasse entsprechend den Anforderungen sowie Ihren eigenen Designüberlegungen. Die Abbildung 7 demonstriert eine mögliche Bildschirmdarstellung im Testlevel `TEST_SWITCH_PULSE`.

Testen Sie nun Ihre neue Implementierung des Tasters mit der Testeinstellung `TEST_LEVEL TEST_SWITCH_PULSE`.

Prüfen Sie dabei auch, ob das automatische Zurücksetzen korrekt funktioniert. Beheben Sie eventuelle Fehler.



Testen Sie die Klasse `SwitchPulse`. Verwenden Sie dazu die Einstellung `TEST_LEVEL TEST_SWITCH_PULSE`.

Wichtig: Ändern Sie auf keinen Fall die Datei `main.cpp`. Ihre Klasse muss die gegebenen Anforderungen erfüllen – dann klappt es auch mit `main.cpp`.



Erzeugen Sie nach dem erfolgreichen Test eine neue Revision und laden Sie das Ergebnis auf den Kursserver hoch.

5 Virtuelle Funktionen und Destruktoren

Nun gilt es, alle Klassen im Verbund zu testen – hierbei werden Sie feststellen, dass Ihre Klassen nicht korrekt zusammenarbeiten.

Verwenden Sie hierfür den Modus `TEST_LEVEL TEST_ALL`. Sie erhalten eine Darstellung ähnlich der in Abbildung 8.

Analysieren Sie den Testcode, der die diversen Schalter erzeugt bzw. darstellt. Betrachten Sie hierbei insbesondere das verwendete Array bzw. den Basistyp mit Sorgfalt. Klären Sie folgende Fragen:

Frage: Warum sieht die Anzeige so defekt aus, wie sie ist? Warum wird offensichtlich die falsche `show`-Funktion verwendet? Bereiten Sie sich darauf vor, dies bei der Abgabe zu erklären.

Die Lösung liegt in der Verwendung virtueller Funktionen. Machen Sie zunächst nur die Funktion `Switch::show` virtuell.

Frage: Warum reicht es nicht, `Switch::show` virtuell zu deklarieren?

Schalten Sie nun alle Funktionen, bei denen Sie dies für nötig halten, auf `virtual` um.

Hinweis: Es wird erwartet, dass Sie Ihre Entscheidung, welche Funktionen `virtual` sein müssen, bei der Abgabe diskutieren können. Notieren Sie sich also Ihre Überlegungen.

Recherche ☕☕

Virtuelle Funktionen



Testen Sie alle Klassen im Verbund.



Analyse des Testprogramms.



Reparieren Sie Ihr Projekt.

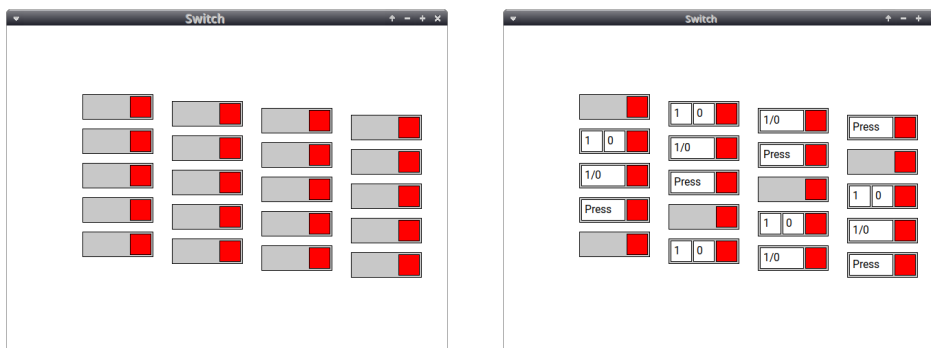


Abbildung 8: Links ist ein fehlgeschlagener Test mit vielen verschiedenen Schaltern zu sehen. Rechts ist das gewünschte Ergebnis nach der Reparatur dargestellt.



Erzeugen Sie nach dem erfolgreichen Test eine neue Revision und laden Sie das Ergebnis auf den Kursserver hoch.

Schalten Sie nun Ihr Programm in den Modus `TEST_DESTRUCTOR`, suchen Sie die Zeile

```
#if TEST_LEVEL == TEST_DESTRUCTOR
```

und setzen Sie einen Breakpoint auf die Anweisung mit dem darauf folgenden `delete`-Operator.

Verwenden Sie nun in bewährter Weise die „step-into“-Funktionalität Ihres Debuggers um herauszufinden, welche Funktionen (insbesondere welche Destruktoren) von welchen Objekten aufgerufen werden.

Schalten Sie nun den Destruktor der Klasse `Switch` auf `virtual` (oder umgekehrt, falls Ihre Destruktor-Implementierung bereits als `virtual` deklariert war).

Wiederholen Sie Ihr Experiment. Was beobachten Sie?

Frage: Versuchen Sie eine Regel zu definieren, wann ein Destruktor unbedingt virtuell sein muss. Begründen Sie auch, warum das so ist.



Zeichnen Sie ein Sequenzdiagramm des `delete`-Operators.